

pyslang 11.0.0 源码编译与离线部署指南

本文用于在无法联网的 Linux 服务器上部署 pyslang。Python 固定为 CPython 3.11，pyslang 源码默认从 PyPI 官方页面手动下载。

参考配置：

```
目标服务器：Linux x86_64, glibc 2.17, CPython 3.11
pyslang: 11.0.0
fmt: 12.1.0
构建镜像：quay.io/pypa/manylinux2014_x86_64
编译器：GCC 11
```

官方 pyslang 11.0.0 的 Linux x86_64 wheel 使用 manylinux_2_27 标签，不能直接用于 glibc 2.17，因此需要在 manylinux2014 环境中重新编译 wheel。

官方资料：

- [pyslang 11.0.0 文件下载页面](#)
- [slang Building & Installation](#)
- [manylinux](#)
- [auditwheel](#)

1. 执行位置

```
[目标服务器]: 最终安装 pyslang 的离线服务器
[Docker 主机]: 可以联网并运行 Docker 的 Linux 主机
[构建容器]: manylinux2014_x86_64 容器内部
```

本文不使用 Conda，Python 操作统一使用 `python -m pip`。

2. 检查目标服务器

以下命令在 [目标服务器] 执行。

2.1 检查系统、架构和 glibc

```
cat /etc/os-release
uname -m
ldd --version | head -n 1
getconf GNU_LIBC_VERSION 2>/dev/null || true
```

要求：

```
架构：x86_64
glibc：2.17 或更高
```

2.2 切换到 Python 3.11

```
type -a python 2>/dev/null || true
type -a python3 2>/dev/null || true
type -a python3.11 2>/dev/null || true
python --version 2>/dev/null || true
python3.11 --version 2>/dev/null || true
```

如果使用 Environment Modules:

```
module avail python 2>&1
module load python/3.11
```

如果直接提供 python3.11, 可创建用户虚拟环境:

```
mkdir -p /path/to/your/home/.venvs
python3.11 -m venv /path/to/your/home/.venvs/pyslang311
source /path/to/your/home/.venvs/pyslang311/bin/activate
```

确认:

```
which python
python --version
python -m pip --version
```

必须显示 Python 3.11.x。

2.3 检查 Python ABI

```
python - <<'PY'
import platform
import sys
import sysconfig
print("Executable:", sys.executable)
print("Version :", sys.version)
print("Impl :", platform.python_implementation())
print("Machine :", platform.machine())
print("SOABI :", sysconfig.get_config_var("SOABI"))
PY
```

目标结果:

```
Impl: CPython
Version: 3.11.x
Machine: x86_64
SOABI: cpython-311-x86_64-linux-gnu
```

3. 手动下载源码

以下操作在联网电脑或 [Docker 主机] 执行。

3.1 创建构建目录

将下面的 BUILD_ROOT 替换为 Docker 主机上的实际目录:

```
export PYSLANG_BUILD_ROOT=/path/to/pyslang_build
mkdir -p /path/to/pyslang_build/src
mkdir -p /path/to/pyslang_build/wheelhouse
mkdir -p /path/to/pyslang_build/repared
mkdir -p /path/to/pyslang_build/offline_bundle
```

3.2 下载 pyslang

打开:

<<https://pypi.org/project/pyslang/11.0.0/#files>>

手动下载:

```
pyslang-11.0.0.tar.gz
```

保存到:

```
/path/to/pyslang_build/src/pyslang-11.0.0.tar.gz
```

需要下载源码包, 不要下载 glibc 2.27 的官方 Linux wheel。

3.3 下载 fmt

打开:

<<https://github.com/fmtlib/fmt/releases/tag/12.1.0>>

手动下载:

```
fmt-12.1.0.tar.gz
```

保存到:

```
/path/to/pyslang_build/src/fmt-12.1.0.tar.gz
```

检查:

```
ls -lh /path/to/pyslang_build/src
```

4. 创建 manylinux 构建容器

以下命令在 [Docker 主机] 执行。

```
docker version
uname -m
df -h
docker pull quay.io/pypa/manylinux2014_x86_64
```

创建容器:

```
docker run -dit \
--name pyslang_builder \
--cpus=8 \
--memory=16g \
-v /path/to/pyslang_build:/workspace \
quay.io/pypa/manylinux2014_x86_64 \
/bin/bash
```

进入容器:

```
docker exec -it pyslang_builder bash
```

检查容器:

```
uname -m
ldd --version | head -n 1
/opt/python/cp311-cp311/bin/python --version
gcc --version | head -n 1
cmake --version
ninja --version
```

5. 安装编译工具

在 [构建容器] 执行:

```
yum install -y \
devtoolset-11-binutils \
devtoolset-11-gcc \
devtoolset-11-gcc-c++ \
export GCC11_ROOT=/opt/rh/devtoolset-11/root/usr
export CC=/opt/rh/devtoolset-11/root/usr/bin/gcc
export CXX=/opt/rh/devtoolset-11/root/usr/bin/g++
export PATH=/opt/rh/devtoolset-11/root/usr/bin:$PATH
which gcc
which g++
gcc --version | head -n 1
g++ --version | head -n 1
```

必须使用 GCC 11。

测试 C++20:

```
cat >/tmp/test_source_location.cpp <<'CPP'
#include <source_location>
#include <iostream>
int main() {
const auto loc = std::source_location::current();
std::cout << loc.file_name() << ":" << loc.line() << "\n";
return 0;
}
CPP
/opt/rh/devtoolset-11/root/usr/bin/g++ \
-std=c++20 \
/tmp/test_source_location.cpp \
-o /tmp/test_source_location
/tmp/test_source_location
```

安装 Python 构建工具:

```
/opt/python/cp311-cp311/bin/python -m pip install --upgrade \
pip setuptools wheel packaging build cmake ninja \
scikit-build-core pybind11 auditwheel patchelf
```

6. 解压源码并应用补丁

```
cd /workspace/src
rm -rf pyslang-11.0.0
rm -rf fmt-12.1.0
tar -xf pyslang-11.0.0.tar.gz
tar -xf fmt-12.1.0.tar.gz
cd /workspace/src/pyslang-11.0.0
cp include/slang/util/Hash.h include/slang/util/Hash.h.original
```

应用 pyslang 11.0.0 补丁：

```
/opt/python/cp311-cp311/bin/python - <<'PY'
from pathlib import Path
path = Path("include/slang/util/Hash.h")
text = path.read_text()
if "#include <filesystem>" not in text:
    marker = "#include <cstring>\n"
    if marker not in text:
        raise RuntimeError("源码版本与 pyslang 11.0.0 不匹配")
    text = text.replace(
        marker,
        "#include <cstring>\n#include <filesystem>\n",
        1,
    )
    specialization = """template<
struct hash<std::filesystem::path> {
    using is_avalanching = void;
    uint64_t operator()(const std::filesystem::path& path) const noexcept {
        return static_cast<uint64_t>(
            std::filesystem::hash_value(path)
        );
    }
};
"""
    insert_before = """template<typename CharT>
struct hash<std::basic_string<CharT>> {"""
    if specialization not in text:
        if insert_before not in text:
            raise RuntimeError("找不到补丁插入位置，源码版本可能不匹配")
        text = text.replace(insert_before, specialization + insert_before, 1)
    path.write_text(text)
    print("Patch applied:", path)
PY
grep -n -A12 -B4 \
'hash<std::filesystem::path>' \
include/slang/util/Hash.h
```

7. 编译 wheel

7.1 设置参数

```
cd /workspace/src/pyslang-11.0.0
export CC=/opt/rh/devtoolset-11/root/usr/bin/gcc
export CXX=/opt/rh/devtoolset-11/root/usr/bin/g++
export PATH=/opt/rh/devtoolset-11/root/usr/bin:$PATH
export LDFLAGS="-static-libstdc++ -static-libgcc"
export CMAKE_GENERATOR=Ninja
export CMAKE_BUILD_PARALLEL_LEVEL=8
export CMAKE_ARGS="\
-DFETCHCONTENT_SOURCE_DIR_FMT=/workspace/src/fmt-12.1.0 \
-DFETCHCONTENT_FULLY_DISCONNECTED=ON"
```

7.2 清理旧产物

```
rm -rf /workspace/build-gcc11
rm -f /workspace/wheelhouse/pyslang-*.whl
rm -f /workspace/repaired/pyslang-*.whl
```

7.3 编译

```
/opt/python/cp311-cp311/bin/python \
-m pip wheel . \
--no-deps \
--no-build-isolation \
--config-settings=build-dir=/workspace/build-gcc11 \
-w /workspace/wheelhouse \
-v
```

成功标准：

```
Successfully built pyslang
```

8. 使用 auditwheel 修复

```
/opt/python/cp311-cp311/bin/auditwheel show \
/workspace/wheelhouse/pyslang-*.whl
mkdir -p /workspace/repaired
rm -f /workspace/repaired/pyslang-*.whl
/opt/python/cp311-cp311/bin/auditwheel repair \
/workspace/wheelhouse/pyslang-*.whl \
--plat manylinux2014_x86_64 \
-w /workspace/repaired
ls -lh /workspace/repaired
/opt/python/cp311-cp311/bin/auditwheel show \
/workspace/repaired/pyslang-*.whl
```

必须确认平台标签包含 manylinux2014_x86_64 或 manylinux_2_17_x86_64，且没有高于 GLIBC_2.17 的依赖。

如果 GLIBC 版本过高，返回第 7 章重新编译。

9. 检查原生扩展

```
rm -rf /tmp/pyslang_wheel_check
mkdir -p /tmp/pyslang_wheel_check
/opt/python/cp311-cp311/bin/python -m zipfile -e \
/workspace/repaired/pyslang-*.whl \
/tmp/pyslang_wheel_check
find /tmp/pyslang_wheel_check \
-type f -name '*.so' \
-exec sh -c '
echo "==== $1 ====="
ldd "$1"
' sh {} \;
```

不能出现 not found。

检查 glibc:

```
find /tmp/pyslang_wheel_check \
-type f -name '*.so' \
-exec sh -c '
readelf --version-info "$1" 2>/dev/null \
| grep -oE "GLIBC_[0-9.]+\" \
| sort -Vu
' sh {} \;
```

最高版本不得超过 GLIBC_2.17。

10. 在干净容器中验证

先退出构建容器:

```
exit
```

在 [Docker 主机] 执行:

```
docker run --rm \
--cpus=2 \
--memory=4g \
-v /path/to/pyslang_build/repared:/wheels:ro \
quay.io/pypa/manylinux2014_x86_64 \
bash -lc '
set -e
rm -rf /tmp/pyslang_site
/opt/python/cp311-cp311/bin/python -m pip install \
--no-index \
--no-deps \
--target /tmp/pyslang_site \
/wheels/pyslang-*.whl
PYTHONPATH=/tmp/pyslang_site \
/opt/python/cp311-cp311/bin/python - <<'PY'
import importlib
import pyslang
from pyslang.syntax import SyntaxTree
native = importlib.import_module("pyslang.pyslang")
tree = SyntaxTree.fromText("""
module test(input logic a, output logic b);
assign b = a;
endmodule
""")
print("pyslang version:", pyslang.__version__)
print("pyslang file :", pyslang.__file__)
print("native file :", native.__file__)
print("diagnostics :", len(tree.diagnostics))
assert len(tree.diagnostics) == 0
print("FRESH MANYLINUX2014 TEST PASSED")
PY
'
```

11. 准备离线安装包

重新进入构建容器：

```
docker start pyslang_builder
docker exec -it pyslang_builder bash
```

在容器内执行：

```
rm -rf /workspace/offline_bundle/*
cp /workspace/repared/pyslang-*.whl /workspace/offline_bundle/
sha256sum /workspace/offline_bundle/*.whl \
| tee /workspace/offline_bundle/SHA256SUMS
exit
```

在 [Docker 主机] 打包：

```
cd /path/to/pyslang_build
tar -czf /path/to/your/home/pyslang11-cp311-manylinux2014-offline.tar.gz \
offline_bundle
```

12. 复制并离线安装

在 [Docker 主机]：

```
scp \  
/path/to/your/home/pyslang11-cp311-manylinux2014-offline.tar.gz \  
用户名@目标服务器:~/
```

在 [目标服务器]:

```
cd /path/to/your/home  
tar -xzf pyslang11-cp311-manylinux2014-offline.tar.gz  
cd /path/to/your/home/offline_bundle  
sha256sum -c SHA256SUMS
```

激活 Python 3.11:

```
module load python/3.11
```

或者:

```
source /path/to/your/python3.11/venv/bin/activate
```

安装:

```
python -m pip install \  
--no-index \  
--no-deps \  
/path/to/your/home/offline_bundle/pyslang-*.whl  
python -m pip show pyslang
```

13. 目标服务器验证

```
python - <<'PY'  
import importlib  
import platform  
import sys  
import pyslang  
from pyslang.syntax import SyntaxTree  
native = importlib.import_module("pyslang.pyslang")  
print("Python executable:", sys.executable)  
print("Python version :", sys.version)  
print("Architecture :", platform.machine())  
print("pyslang version :", pyslang.__version__)  
print("pyslang package :", pyslang.__file__)  
print("native extension :", native.__file__)  
tree = SyntaxTree.fromText("""  
module test #(  
parameter int WIDTH = 8  
)(  
input logic [WIDTH-1:0] data_in,  
output logic [WIDTH-1:0] data_out  
);  
assign data_out = data_in;  
endmodule  
""")  
print("diagnostics:", len(tree.diagnostics))  
assert len(tree.diagnostics) == 0  
print("TARGET SERVER TEST PASSED")  
PY
```

必须看到 TARGET SERVER TEST PASSED。

检查动态依赖：

```
python - <<'PY'
import importlib
print(importlib.import_module("pyslang.pyslang").__file__)
PY
```

将上一步输出的 .so 路径代入：

```
ldd /path/to/pyslang.pyslang.so
readelf --version-info /path/to/pyslang.pyslang.so 2>/dev/null \
| grep -oE 'GLIBC_[0-9.]+' \
| sort -Vu
```

不能出现 not found，最高 GLIBC 版本不得超过 GLIBC_2.17。

14. 常见错误

source_location 错误

检查是否实际使用 GCC 11：

```
which g++
g++ --version
```

修改编译器后删除 /workspace/build-gcc11，重新编译。

filesystem hash 错误

检查 Hash.h 补丁：

```
grep -n -A12 -B4 \
'hash<std::filesystem::path>' \
include/slang/util/Hash.h
```

构建时访问 GitHub

检查 CMake 参数：

```
echo "$CMAKE_ARGS"
```

必须包含本地 fmt 路径和 FETCHCONTENT_FULLY_DISCONNECTED=ON。

wheel 不支持当前平台

检查：

```
python --version
uname -m
ldd --version | head -n 1
python -m pip debug --verbose
```

本文 wheel 要求 CPython 3.11、x86_64、glibc 不低于 2.17。

GLIBCXX_x.x.xx not found

确认编译时设置：

```
export LDFLAGS="-static-libstdc++ -static-libgcc"
```

清理构建目录后重新编译，并重新执行原生扩展检查。

pyslang.SyntaxTree 不存在

使用：

```
from pyslang.syntax import SyntaxTree
```

15. 环境变化时需要修改的内容

Python 固定为 3.11，只关注以下变化：

变化	需要修改
glibc 变化	manylinux 镜像和 auditwheel 平台标签
CPU 架构变化	Docker 镜像后缀和构建主机架构
pyslang 版本变化	源码、依赖、补丁和 API 测试
GCC 或 CMake 变化	编译器路径和版本检查

重新编译前确认：

1. Python 仍然是 CPython 3.11；
2. 架构与目标服务器一致；
3. 构建环境的 glibc 不高于目标基线；
4. 当前源码的 CMake、C++ 和依赖要求；
5. 补丁仍适用于当前源码；
6. auditwheel 和目标服务器验证重新通过。

16. 最终验收清单

```
[ ] 目标服务器使用 CPython 3.11
[ ] 目标服务器架构为 x86_64
[ ] wheel SHA256 校验通过
[ ] wheel 标签能够被目标 pip 接受
[ ] auditwheel 检查通过
[ ] 最高 GLIBC 符号不超过 GLIBC_2.17
[ ] ldd 没有出现 not found
[ ] import pyslang 成功
[ ] from pyslang.syntax import SyntaxTree 成功
[ ] SystemVerilog 解析无诊断错误
[ ] 目标服务器测试输出 TARGET SERVER TEST PASSED
```

确认完成后，可以删除构建容器：

```
docker stop pyslang_builder
docker rm pyslang_builder
```